

Basi di Dati

Nicolò Giuliani

Indice

1	Introduzione	3
1.1	Sistema informativo e sistema informatico	3
1.2	Informazione e dati	3
1.3	DBMS	3
1.4	Transazioni	4
1.5	Evoluzione storica	5
1.6	DBMS e file system	5
1.7	Modelli dei dati	5
1.8	Architettura di un DBMS	6
1.9	Linguaggi per basi di dati	7
1.10	Attori e utenti	7
1.11	Pregi e difetti dei DBMS	8
2	Il modello relazionale	8
2.1	Modelli logici	8
2.2	Il modello relazionale	8
2.3	Relazione logica	9
2.4	Struttura non posizionale	9
2.5	Modello value based	9
2.6	Schemi e istanze	10
2.7	Casi particolari	10
2.8	Informazione parziale	10
2.9	Vincoli di integrità	11
2.10	Vincoli di tupla	11
2.11	Chiavi	12
2.12	Chiavi e valori nulli	13
2.13	Integrità referenziale	13
2.14	Azioni compensative	13
3	Algebra relazionale e calcolo relazionale	14
3.1	Linguaggi di interrogazione	14
3.2	Algebra relazionale	14
3.3	Operatori insiemistici	15
3.4	Ridenominazione	15
3.5	Selezione	15
3.6	Proiezione	16
3.7	Selezione e proiezione	16
3.8	Join naturale	16
3.9	Cardinalità del join	17

3.10	Join esterno	18
3.11	Prodotto cartesiano e theta-join	18
3.12	Esempi di interrogazioni	19
3.13	Equivalenza di espressioni	19
3.14	Selezione con valori nulli	21
3.15	Viste	21
3.16	Notazione alternativa per i join	22
3.17	Calcolo relazionale	22
3.18	Calcolo su domini	23
3.19	Calcolo su tuple con dichiarazioni di range	25
3.20	Limiti di algebra e calcolo	26
3.21	Datalog	26

1 Introduzione

1.1 Sistema informativo e sistema informatico

Base di dati (significato generico): insieme organizzato di dati che supporta lo svolgimento delle attività specifiche di un ente (istituzione, azienda, ufficio, persona).

Base di dati (significato specifico): insieme di dati gestito da un DBMS.

Sistema informativo

componente di un ente che gestisce le informazioni di interesse. Ogni ente ha il proprio sistema informativo, anche se non esplicitamente identificabile nella sua organizzazione; va studiato nel suo contesto operativo. È un concetto *indipendente* dall'automazione: esistono da secoli organizzazioni la cui unica missione è gestire dati (anagrafi, banche).

Sistema informatico

la porzione automatizzata del sistema informativo, cioè la componente che gestisce le informazioni con tecnologie informatiche.

Gerarchia di inclusione: Azienda $\supset$ Organizzazione $\supset$ Sistema informativo $\supset$ Sistema informatico.

Gestione dell'informazione: raccolta/acquisizione, archiviazione/conservazione, elaborazione/trasformazione/produzione, distribuzione/comunicazione/scambio.

1.2 Informazione e dati

Le attività umane rappresentano l'informazione in molti modi (idee informali, linguaggio naturale, disegni e diagrammi, numeri e codici) su supporti diversi (mente umana, carta, dispositivi elettronici). Nei sistemi informatici invece l'informazione è rappresentata in modo povero ma essenziale: come **dati**.

- *informazione*: fatti forniti o appresi su qualcosa o qualcuno.
- *dato*: elemento di informazione, punto di partenza fissato di un procedimento.

Il dato è il risultato del processo di organizzazione, codifica e memorizzazione dell'informazione. **Il dato da solo è inutile senza la sua interpretazione** (es. i numeri su un cartello stradale non dicono nulla se non si sa che sono orari, e quale orario vale in quale giorno).

Perché i dati? È difficile rappresentare con precisione informazione e conoscenza ricche. I dati sono una *risorsa strategica* perché sono più stabili delle altre componenti (processi aziendali, tecnologie, ruoli): es. i dati di banche e anagrafi.

1.3 DBMS

DBMS (DataBase Management System): sistema che gestisce collezioni di dati

- **grandi**
- **persistenti**
- **condivise**

garantendo

- **privatezza**
- **affidabilità**
- **efficienza**
- **efficacia**

Esempi: DB2, Oracle, SQLServer, MySQL, PostgreSQL, Access, BigQuery, SQLite.

Grandi dimensioni molto maggiori della memoria centrale; il solo limite considerato è quello fisico dei dispositivi. Ordini di grandezza: 1–5 TB (dati transazionali), 30–50 TB (dati decisionali), 500–800 TB (dati scientifici), 10^{11} record.

Persistenti il loro tempo di vita è indipendente da quello dei processi che li usano.

Condivise ogni (grande) azienda è divisa in settori, ognuno con un proprio sotto-sistema informativo. Senza condivisione nascono **ridondanza** (stessi dati ripetuti) e quindi possibile **incoerenza** (descrizioni diverse non coincidono). La base di dati è una risorsa *integrata e condivisa*, da cui la necessità di **meccanismi di autorizzazione** (attività diverse sugli stessi dati) e **controllo della concorrenza** (più utenti sugli stessi dati).

Privatezza si definiscono autorizzazioni del tipo “l’utente A può leggere tutti i dati e modificare il dato X”.

Affidabilità resistenza a malfunzionamenti hardware e software. La base di dati è una risorsa preziosa e va conservata a lungo termine. Tecnica fondamentale: la gestione delle **transazioni**.

Efficienza uso proficuo delle risorse di memoria (primaria e secondaria) e di tempo (esecuzione e risposta). Poiché i DBMS offrono moltissime funzioni, potrebbero risultare inefficienti: da qui grandi investimenti e concorrenza fra prodotti. L’efficienza è anche frutto della qualità del software.

Efficacia rendere produttive le attività degli utenti, offrendo funzionalità potenti e flessibili.

1.4 Transazioni

Una **transazione** è un insieme di operazioni non separabili (*atomiche*), corrette anche in presenza di concorrenza, con effetti definitivi.

Atomicità

o si esegue tutta la sequenza o non se ne esegue nulla. Es.: “sposta denaro dal conto A al conto B: o si preleva da A e si versa su B, o non si fa nulla”.

Concorrenza

transazioni concorrenti devono restare coerenti: due assegni sullo stesso conto incassati contemporaneamente non devono perdersi; due agenzie non devono prenotare due volte lo stesso posto sul treno.

Durabilità

il **commit** garantisce che il risultato finale sia registrato e conservato, anche in caso di concorrenza o guasti.

Due punti di vista sulle transazioni:

- *utente*: un programma che realizza una funzione utile, frequente e predefinita, con poche eccezioni previste (versamento in conto, emissione di un certificato di nascita, iscrizione all'anagrafe, prenotazione di un volo). Sono scritte in un linguaggio ospite, noto o *ad hoc*.
- *sistema*: sequenza indivisibile di operazioni (cfr. affidabilità).

1.5 Evoluzione storica

Anni '70 — nessun DBMS

le applicazioni accedono direttamente ai file del sistema operativo; i dati comuni vengono duplicati nelle applicazioni.

Anni '80 — primi DBMS

fra le applicazioni (Pascal, C) e i file si interpone il DBMS, che offre tabelle.

Anni '90 — DBMS attivi

il comportamento procedurale comune viene condiviso fra le applicazioni. Nascono le **stored procedures**, che però non sono standardizzate e soffrono di *impedance mismatch* con il linguaggio usato per esprimerle; si introducono quindi i **trigger**, regole che modellano il comportamento procedurale condiviso e sono gestite dal DBMS stesso.

Anni 2000

architettura client/server su Internet: front-end (Javascript) sul client, Web Application Server e back-end (Java), database attivo con tabelle e trigger sul server.

Anni 2010

app mobili: front-end in Java, Objective-C, HTML5; il resto dell'architettura resta invariato.

1.6 DBMS e file system

Anche un file system può gestire grandi collezioni di dati, ma offre un accesso a grana grossa: “tutto o niente”. I DBMS estendono le funzionalità del file system, offrendo più servizi in modo integrato.

Descrizione dei dati ogni programma che non usa un DBMS contiene al proprio interno la descrizione della struttura dei file che tratta: programmi diversi possono avere descrizioni diverse, generando incoerenza fra le rappresentazioni. Nei DBMS una porzione della base di dati (il **catalogo** o *dizionario*) contiene una descrizione centralizzata dei dati, condivisa da tutte le applicazioni.

1.7 Modelli dei dati

Nei DBMS i dati sono descritti a diversi **livelli di astrazione**: i programmi usano una rappresentazione di alto livello indipendente da quella di livello più basso, così una modifica al livello basso non richiede di modificare i programmi. Questo è ottenuto attraverso il concetto di **modello dei dati**.

Modello dei dati: insieme di costrutti utilizzati per organizzare i dati e descriverne il comportamento. Componente fondamentale: dare struttura ai dati tramite **costruttori di tipo**. Es.: il **modello relazionale** fornisce il costruttore *relazione*, che permette di definire insiemi di record omogenei.

Schema e istanza in ogni base di dati esistono

- lo **schema**: invariante nel tempo, descrive la struttura dei dati (aspetto *intensionale*) — es. l'intestazione delle tabelle;
- l'**istanza**: i valori effettivi, che possono cambiare rapidamente (aspetto *estensionale*) — es. il corpo delle tabelle.

Due tipologie di modelli

Modelli logici

usati dai DBMS per organizzare e memorizzare i dati; usati dai programmi, indipendenti dalla rappresentazione fisica. Es.: relazionale, reticolare, gerarchico, a oggetti, XML.

Modelli concettuali

rappresentano i dati indipendentemente da ogni sistema specifico; descrivono concetti del mondo reale e si usano nella fase preliminare di progettazione. Il più diffuso è **Entity-Relationship**.

1.8 Architettura di un DBMS

Architettura semplificata (a due livelli): utente $\rightarrow$ *schema logico* $\rightarrow$ *schema interno* $\rightarrow$ DB.

Schema logico

descrive la struttura della base di dati tramite il modello logico dei dati (es. la struttura delle tabelle).

Schema interno (o fisico)

rappresenta lo schema logico a livello di memorizzazione, con strutture dati *raw* (file, record, puntatori).

Poiché la rappresentazione logica è indipendente da quella fisica — una tabella si usa “così com'è”, indipendentemente dalla rappresentazione sottostante, che può anche variare nel tempo — nel corso si considera solo il livello logico.

Architettura standard ANSI/SPARC a tre livelli aggiunge sopra lo schema logico uno o più **schemi esterni**, ciascuno usato da un gruppo di utenti:

- **schema logico**: struttura della base di dati secondo il modello logico “principale”;
- **schema interno**: rappresentazione a livello di memorizzazione;
- **schema esterno**: descrive una parte della base di dati in un modello logico (*viste* parziali, tabelle derivate, anche da modelli diversi).

Indipendenza dei dati è conseguenza della stratificazione; l'accesso ai dati avviene sempre a livello esterno (che può coincidere con quello logico).

Indipendenza fisica

livello logico ed esterno sono indipendenti dal livello fisico: una relazione è usata allo stesso modo qualunque sia la sua realizzazione fisica, che può cambiare senza modificare i programmi.

Indipendenza logica

il livello esterno è indipendente da quello logico: si può operare su una vista senza modificare il livello logico, e modifiche “trasparenti” allo schema logico lasciano inalterato lo schema esterno.

1.9 Linguaggi per basi di dati

Un altro aspetto dell'efficacia è la disponibilità di linguaggi e interfacce diverse:

- linguaggi testuali *interattivi* (**SQL**);
- comandi SQL immersi in un linguaggio **ospite** (Pascal, Java, C);
- comandi SQL immersi in un linguaggio *ad hoc* con altre funzionalità (grafici, stampa di tabelle) — es. Oracle PL/SQL;
- interfacce amichevoli, senza linguaggio testuale.

Esempio SQL interattivo date le tabelle `Corsi(Corso, Docente, Aula)` e `Aule(Nome, Edificio, Piano)`, per ottenere i corsi che si tengono al piano terra:

```
SELECT Corso, Aula, Piano
FROM Aule, Corsi
WHERE Nome = Aula
      AND Piano = 'Terra'
```

Separazione fra dati e programmi

DML

data manipulation language: interrogazione e aggiornamento delle *istanze* della base di dati.

DDL

data definition language: definizione degli *schemi* (logico, esterno, fisico) e altre operazioni. Es.:

```
CREATE TABLE orario (
  corso    CHAR(20),
  docente  CHAR(20),
  aula     CHAR(4),
  ora      CHAR(5) )
```

1.10 Attori e utenti

- progettisti e realizzatori di DBMS;
- progettisti della base di dati e amministratori (**DBA**);
- progettisti e programmatori di applicazioni;
- utenti:
 - *utenti finali* (terminalisti): eseguono operazioni predefinite (transazioni);
 - *utenti casuali*: eseguono operazioni non previste a priori, usando linguaggi interattivi.

DBA persona o gruppo responsabile del controllo e della gestione centralizzata della base di dati (efficienza, affidabilità, autorizzazioni). Spesso il DBA si occupa anche della progettazione, tranne che nei progetti complessi.

1.11 Pregi e difetti dei DBMS

Pregi

- dati come risorsa comune, base di dati come modello della realtà;
- gestione dei dati centralizzata, standardizzabile ed economia di scala;
- disponibilità di servizi integrati;
- riduzione di ridondanze e incoerenze;
- indipendenza dei dati (favorisce sviluppo e manutenzione delle applicazioni).

Difetti

- costo dei prodotti e della loro adozione;
- funzionalità non separabili (con conseguente riduzione di efficienza).

2 Il modello relazionale

2.1 Modelli logici

Esistono tre modelli logici *tradizionali*:

- **gerarchico**;
- **reticolare** (network);
- **relazionale**.

Più recenti sono il modello a *oggetti* (poco diffuso) e i modelli su *XML* (considerati complementari al relazionale).

Caratteristica distintiva

Gerarchico e reticolare

usano **riferimenti espliciti** (puntatori) fra i record.

Relazionale

è **value based**: i riferimenti fra dati memorizzati in relazioni diverse sono rappresentati tramite *valori* (non puntatori).

2.2 Il modello relazionale

Definito da **E. F. Codd** nel 1970 per garantire l'indipendenza dei dati. Implementato in DBMS reali solo a partire dal 1981: realizzare indipendenza dei dati in modo efficiente e affidabile non è banale. Il modello si basa sulla definizione logica di *relazione* (con alcune differenze) e le relazioni sono rappresentate tramite **tabelle**.

Relazione, relation, relationship tre concetti da tenere distinti:

- *relazione logica*: nel senso della teoria degli insiemi;
- *relazione* nel modello relazionale;
- *relationship*: esprime una specifica classe di fatti nel modello Entity-Relationship.

2.3 Relazione logica

Dati n insiemi $D_1, \dots, D_n$ (non necessariamente distinti):

- il **prodotto cartesiano** $D_1 \times \dots \times D_n$ è l'insieme di tutte le n -uple $(d_1, \dots, d_n)$ tali che $d_1 \in D_1, \dots, d_n \in D_n$;
- una **relazione logica** su $D_1, \dots, D_n$ è un *sottoinsieme* $r \subseteq D_1 \times \dots \times D_n$;
- i D_i sono i **domini** della relazione.

Proprietà

- è un *insieme* di n -uple: non c'è ordine fra le tuple;
- le tuple sono tutte *distinte*;
- ogni n -upla è *ordinata*: l' i -esimo valore proviene dall' i -esimo dominio.

Struttura posizionale in `Matches` $\subseteq \text{string} \times \text{string} \times \text{int} \times \text{int}$ lo stesso dominio compare con **ruoli** diversi, distinti solo dalla *posizione* (1^a colonna = squadra di casa, 2^a = ospite, ecc.). Questa è una struttura **posizionale**.

2.4 Struttura non posizionale

Nel modello relazionale a ciascun dominio si associa un **attributo**, un nome unico che ne esprime il “ruolo”. La posizione delle colonne diventa irrilevante: la struttura è **non posizionale**. Es. (`Home`, `Away`, `GoalsH`, `GoalsA`): scambiando l'ordine delle colonne (con le relative intestazioni) l'informazione non cambia; scambiare *solo* le intestazioni cambia invece il significato.

Tabelle e relazioni una tabella rappresenta una relazione se

- tutte le righe sono *diverse*;
- tutte le intestazioni di colonna sono *diverse*;
- i valori all'interno di ogni colonna sono *omogenei* (stesso dominio).

Righe e colonne possono assumere qualunque posizione.

2.5 Modello value based

I riferimenti fra dati in relazioni diverse sono espressi tramite **valori** nelle tuple (es. `Exams.Student = Students.Number`), non tramite puntatori. Altri modelli (reticolare, gerarchico, a oggetti) usano invece riferimenti espliciti gestiti dal sistema.

Pregi del modello value based

- indipendenza dalla struttura fisica, che può cambiare dinamicamente (una cosa analoga si può ottenere con puntatori “di alto livello”);
- si memorizza solo ciò che è rilevante dal punto di vista applicativo;
- utente e programmatore vedono gli stessi dati;
- i dati possono essere facilmente condivisi fra ambienti diversi;
- i puntatori sono direzionali, i valori no.

2.6 Schemi e istanze

Schema di una relazione un nome di relazione R con un insieme di attributi $A_1, \dots, A_n$:

$$R(A_1, \dots, A_n)$$

Es. STUDENTS(Number, Surname, Name, Year of Birth).

Schema di database relazionale insieme di schemi di relazione:

$$\mathcal{R} = \{R_1(X_1), \dots, R_k(X_k)\}$$

Tupla una tupla su un insieme di attributi X è un **mapping** che associa a ogni attributo $A \in X$ un valore nel dominio di A . Con $t[A]$ si indica il valore della tupla t sull'attributo A (es. $t[\text{Name}] = \text{Mario}$).

Istanze

- una **istanza di relazione** r è un insieme finito di tuple con schema $R(X)$;
- una **istanza di database relazionale** su $\mathcal{R} = \{R_1(X_1), \dots, R_n(X_n)\}$ è un insieme $r = \{r_1, \dots, r_n\}$ di istanze di relazione (r_i istanza di R_i).

2.7 Casi particolari

Relazioni con un solo attributo sono ammesse: es. WorkerStudents(Number), che elenca soltanto le matricole degli studenti lavoratori.

Strutture annidate molti dati reali (es. scontrini, fatture) sono naturalmente *annidati*: un'intestazione con vari campi ripetuti (righe di prodotto). Il modello relazionale è *piatto*: le strutture annidate si rappresentano con più relazioni collegate da valori. Es. Receipts(Number, Date, Total) e Courses(Receipt, Row, No, Description, Price). La rappresentazione scelta dipende dagli obiettivi: righe ripetute ammesse? ordine delle righe significativo? esistono più rappresentazioni possibili.

2.8 Informazione parziale

Il modello relazionale ha una struttura *rigida*: l'informazione è espressa in tuple che devono rispettare uno schema. Ma i dati reali possono non combaciare con il formato atteso (es. un "secondo nome" non sempre esistente).

Perché non usare valori speciali scegliere un valore del dominio (0, stringa vuota, "99", ...) per marcare l'assenza è pericoloso:

- potrebbero non esserci valori "inutilizzati" disponibili;
- quei valori potrebbero in seguito diventare significativi;
- bisognerebbe comunque tracciare a livello di software questi valori "non rappresentabili".

Valore NULL tecnica grezza ma efficace: introdurre un valore speciale **NULL** che indica l'assenza di un valore atteso e non appartiene al dominio. Formalmente, per ogni attributo A , $t[A]$ è o un valore in $\text{dom}(A)$ oppure **NULL**. Servono vincoli specifici per stabilire quando **NULL** non è ammesso.

Tipi di NULL ci sono almeno tre casi diversi di valore mancante:

- valore *sconosciuto* (esiste ma non lo conosciamo);
- valore *inesistente* (non esiste per quella tupla);
- valore *non informativo* (non sappiamo se esiste).

I DBMS moderni non distinguono fra questi casi.

Troppi NULL una istanza con troppi NULL (specialmente nelle chiavi o nei riferimenti) perde valore informativo: l'uso di NULL va limitato.

2.9 Vincoli di integrità

Alcune istanze sintatticamente corrette possono non rappresentare informazione corretta per l'applicazione (es. un voto "32", una *laude* con voto < 30 , un riferimento a uno studente inesistente).

Vincolo di integrità proprietà che deve valere sulle istanze per rappresentare informazione corretta. Formalmente è una funzione booleana (**predicato**) che associa a ogni istanza *true* o *false*.

Perché

- descrizione più accurata del dominio reale;
- supporto alla *data quality*;
- utili in fase di progettazione;
- usati nella valutazione delle interrogazioni.

Osservazione i DBMS non supportano tutti i tipi di vincolo: si dichiarano quelli supportati e il DBMS ne impedisce la violazione; per gli altri il programmatore deve implementarli fuori dal DBMS.

Classificazione

Intra-relazionali

coinvolgono una sola relazione:

- su valori (*domain constraints*);
- su tuple.

Inter-relazionali

coinvolgono più relazioni.

2.10 Vincoli di tupla

Esprimono regole sui valori di ogni singola tupla, indipendentemente dalle altre. Caso specifico: i **domain constraints** coinvolgono un solo attributo.

Sintassi espressione booleana di atomi che confrontano:

- valori del dominio degli attributi, es. $(\text{Grade} \geq 18) \text{ AND } (\text{Grade} \leq 30)$;
- espressioni aritmetiche su tali valori, es. $\text{GrossPay} = (\text{Deductions} + \text{Net})$.

Una tupla (Bruni, 50.000, 11.000, 36.000) viola il vincolo perché $11.000 + 36.000 \neq 50.000$.

2.11 Chiavi

Nel mondo reale ogni entità è identificabile: nella tabella **Students** ogni tupla ha un **Number** unico e non esistono tuple con stessi **Surname**, **Name**, **Date of Birth**. Formalizziamo:

Superchiave

un insieme K di attributi di r tale che non esistono due tuple distinte $t_1, t_2 \in r$ con $t_1[K] = t_2[K]$.

Chiave

una superchiave *minimale*: nessun suo sottoinsieme proprio è già superchiave.

Esempi in **Students**(**Number**, **Surname**, **Name**, **Curriculum**, **Birth**):

- **Number** è chiave: è superchiave (identifica ciascuna tupla) e contiene un solo attributo, quindi è minimale;
- {**Surname**, **Name**, **Birth**} è un'altra chiave: è superchiave ed è minimale;
- {**Surname**, **Curriculum**} potrebbe essere superchiave in un'istanza particolare ma non nel caso generale.

Vincoli, schema e stati

- i vincoli corrispondono a proprietà del mondo reale, modellate nel DB;
- sono modellati sullo *schema* (si applicano a tutte le tuple);
- ogni schema può avere un insieme di vincoli: le tuple memorizzate sono **corrette** (valide) se soddisfano tutti i vincoli;
- un'istanza può soddisfare *per caso* altri vincoli non dichiarati.

Esistenza delle chiavi ogni relazione:

- non può contenere tuple con tutti i valori uguali (è un insieme);
- ha come superchiave l'insieme di tutti i suoi attributi;
- quindi ha almeno una chiave.

Importanza

- l'esistenza di almeno una chiave garantisce l'*accessibilità* di ogni tupla;
- le chiavi permettono di correlare tuple fra relazioni diverse: il modello relazionale è *value based*.

2.12 Chiavi e valori nulli

Se una tupla contiene NULL sugli attributi della chiave, i suoi valori non permettono di:

- identificare la tupla;
- creare facilmente riferimenti esterni da altre relazioni.

L'uso di NULL sulle chiavi va limitato.

Chiave primaria fra le chiavi di una relazione se ne sceglie una (**primary key**) che *non ammette valori NULL*. Convenzionalmente si indica con la sottolineatura degli attributi nello schema.

2.13 Integrità referenziale

Le informazioni fra relazioni diverse sono correlate tramite valori comuni, in particolare valori di chiavi (primarie). Tali correlazioni devono essere *coerenti*: es. il codice del vigile in `Infringement.Policeman` deve corrispondere a un ID esistente in `Policemen`.

Vincolo di integrità referenziale (*foreign key*) fra gli attributi X di una relazione R_1 e una relazione R_2 : i valori di X in R_1 devono comparire come valori della chiave primaria di R_2 .

Esempio in un DB con `Infringement(Code, Date, Policeman, Country, Plate)`, `Policemen(ID, Surname, Name)`, `Car(Country, Plate, Surname, Name)`:

- $\text{Policeman} \rightarrow \text{Policemen}(\text{ID})$;
- $(\text{Country}, \text{Plate}) \text{ in } \text{Infringement} \rightarrow \text{Car}(\text{Country}, \text{Plate})$.

Una tupla `Infringement` con `(Italy, AJ046EZ)`, mentre in `Car` `AJ046EZ` appartiene alla `France`, viola il vincolo.

Osservazioni

- ruolo cruciale nel modello value based;
- i vincoli possono essere *rilassati* in presenza di NULL;
- le violazioni causate da operazioni possono essere gestite tramite *azioni compensative*;
- attenzione ai vincoli definiti su più attributi: l'*ordine* degli attributi è rilevante (in `Crashes(Code, CountryA, PlateA, CountryB, PlateB)` servono due vincoli distinti verso `Car(Country, Plate)`, uno per (A, A) e uno per (B, B)).

2.14 Azioni compensative

Quando una modifica (es. cancellazione di una tupla referenziata) violerebbe un vincolo referenziale, il DBMS può reagire in modi diversi.

Comportamento standard

Restrict

rifiuta la cancellazione.

Azioni compensative possibili

Cascade

rimuove “a catena” le tuple referenzianti (es. eliminando il progetto XYZ da `Projects` si eliminano anche i dipendenti in `Employees` con `Project = XYZ`).

Set NULL

pone NULL nell'attributo referenziante (`Employees.Project` diventa NULL per chi lavorava a XYZ).

Set default

introduce un valore di default.

3 Algebra relazionale e calcolo relazionale

3.1 Linguaggi di interrogazione

I linguaggi per basi di dati operano sullo schema (**DDL**) o sui dati (**DML**); il DML comprende istruzioni di *interrogazione* (query) e di *aggiornamento* (inserimento e modifica dei dati).

Dichiarativi

specificano *che cosa* si vuole ottenere, cioè le proprietà del risultato.

Procedurali

specificano *come* il risultato viene ottenuto.

- **Algebra relazionale**: procedurale;
- **Calcolo relazionale**: dichiarativo (teorico, non implementato);
- **SQL** (Structured Query Language): parzialmente dichiarativo (implementato);
- **QBE** (Query by Example): dichiarativo (implementato).

3.2 Algebra relazionale

È definita da un insieme di **operatori** che agiscono su relazioni, producono relazioni come risultato e sono quindi *componibili* fra loro.

Operatori:

- unione, intersezione, differenza;
- ridenominazione (ρ);
- selezione (σ);
- proiezione (π);
- join (join naturale, prodotto cartesiano, theta-join).

3.3 Operatori insiemistici

Le relazioni sono insiemi e anche i risultati devono esserlo: si possono usare **unione** ($\cup$), **intersezione** ($\cap$) e **differenza** ($-$), ma solo fra relazioni con lo *stesso schema*.

Graduated			Technicians		
Number	Name	Age	Number	Name	Age
7274	Rossi	42	9297	Neri	33
7432	Neri	54	7432	Neri	54
9824	Verdi	45	9824	Verdi	45

- Graduated $\cup$ Technicians: 4 tuple (7274, 7432, 9824, 9297) — le tuple comuni compaiono una volta sola;
- Graduated $\cap$ Technicians: le tuple 7432 e 9824;
- Graduated $-$ Technicians: la sola tupla 7274.

Un'unione sensata ma impossibile l'unione di Fatherhood (attributi Father, Child) e Motherhood (attributi Mother, Child) avrebbe senso (l'insieme dei genitori) ma gli schemi sono diversi: serve prima una ridenominazione.

3.4 Ridenominazione

Operatore *unario*: “cambia lo schema” lasciando inalterati i dati.

$$\rho_{\text{NewName} \leftarrow \text{OldName}}(\text{Relation})$$

rinomina l'attributo OldName in NewName. Si possono rinominare più attributi insieme.

Esempi

$$\rho_{\text{Parent} \leftarrow \text{Father}}(\text{Fatherhood}) \cup \rho_{\text{Parent} \leftarrow \text{Mother}}(\text{Motherhood})$$

produce una relazione (Parent, Child) con tutte le coppie genitore-figlio. Analogamente, con Employee(Surname, Office, Salary) e Worker(Surname, Factory, Wage):

$$\rho_{\text{Office, Pay} \leftarrow \text{Office, Salary}}(\text{Employee}) \cup \rho_{\text{Office, Pay} \leftarrow \text{Factory, Wage}}(\text{Worker})$$

ha schema (Surname, Office, Pay).

3.5 Selezione

Operatore *unario*. Il risultato:

- ha lo *stesso schema* dell'operando;
- contiene un *sottoinsieme* delle tuple dell'operando;
- contiene le sole tuple che soddisfano un **predicato**.

$$\sigma_{\text{Predicate}}(\text{Relation})$$

dove *Predicate* è un'espressione booleana valutata su ogni tupla della relazione.

Esempi su `Employee(Number, Surname, Office, Salary)`:

- chi guadagna più di 50: $\sigma_{\text{Salary}>50}(\text{Employee})$;
- chi guadagna più di 50 e lavora a Milano: $\sigma_{\text{Salary}>50 \wedge \text{Office}='Milan'}(\text{Employee})$;
- chi ha il cognome uguale al nome della sede (confronto fra due attributi della stessa tupla): $\sigma_{\text{Surname}=\text{Office}}(\text{Employee})$.

3.6 Proiezione

Operatore *unario*. Il risultato:

- ha come schema un *sottoinsieme* degli attributi dell'operando;
- è costruito a partire da *tutte* le tuple dell'operando.

$$\pi_{\text{AttributeList}}(\text{Relation})$$

contiene tutte le tuple della relazione, ristrette agli attributi in *AttributeList*.

Cardinalità il risultato di una proiezione

- contiene *al più* tante tuple quante l'operando;
- può contenerne meno: restringendo gli attributi alcune tuple possono diventare uguali, e i duplicati vengono eliminati (una relazione è un insieme);
- se X è una **superchiave** di R , allora $\pi_X(R)$ contiene esattamente tante tuple quante R .

Es.: se `Employee` contiene due **Rossi** entrambi a **Rome**, $\pi_{\text{Surname,Office}}(\text{Employee})$ ha una tupla in meno; $\pi_{\text{Number,Surname}}(\text{Employee})$ invece le conserva tutte perché **Number** è chiave.

3.7 Selezione e proiezione

Sono operatori “ortogonali”:

- **selezione**: decomposizione *orizzontale* (sceglie righe);
- **proiezione**: decomposizione *verticale* (sceglie colonne).

Combinandole si estrae da una relazione solo l'informazione di interesse:

$$\pi_{\text{Number,Surname}}(\sigma_{\text{Salary}>50}(\text{Employee}))$$

Limite con questi soli operatori si estrae informazione da *una sola* relazione: non si possono correlare tuple di relazioni diverse, né tuple diverse della stessa relazione.

3.8 Join naturale

Il **join** è l'operatore più interessante dell'algebra relazionale: permette di correlare tuple di relazioni diverse. Es. in un concorso i compiti sono anonimi e associati a una busta chiusa col nome del candidato; compito e busta hanno lo stesso ID: da `(ID, Grade)` e `(ID, Candidate)` si ottiene `(ID, Candidate, Grade)`.

Il join naturale è un operatore *binario* (generalizzabile a n operandi per associatività). Il risultato:

- ha come schema l'*unione* degli attributi dei due operandi;
- ogni sua tupla è ottenuta combinando due tuple, una per operando, che coincidono sugli attributi comuni.

Definizione dati $R_1(X_1)$ e $R_2(X_2)$, $R_1 \bowtie R_2$ è una relazione su X_1X_2 :

$$R_1 \bowtie R_2 = \{ t \text{ su } X_1X_2 \mid \exists t_1 \in R_1, t_2 \in R_2 \text{ con } t[X_1] = t_1 \text{ e } t[X_2] = t_2 \}$$

Casi

Employee	Dept		Dept	Chief	=	Employee	Dept	Chief
Rossi	A	$\bowtie$	A	Mori		Rossi	A	Mori
Neri	B		B	Bruni		Neri	B	Bruni
Bianchi	B					Bianchi	B	Bruni

Join completo

ogni tupla di entrambi gli operandi contribuisce al risultato (esempio sopra).

Join non completo

alcune tuple non trovano corrispondenza e restano escluse (tuple *dangling*): con **Dept** $\in \{B, C\}$ nel secondo operando, **Rossi/A** e il dipartimento **C** spariscono.

Join vuoto

nessuna tupla trova corrispondenza (dipartimenti **D**, **C** a destra): il risultato ha lo schema ma nessuna tupla.

Join con $n \times m$ tuple

se tutte le tuple hanno lo stesso valore sull'attributo comune, ogni tupla si combina con tutte quelle dell'altro operando.

3.9 Cardinalità del join

- In generale:

$$0 \leq |R_1 \bowtie R_2| \leq |R_1| \times |R_2|$$

- se il join coinvolge una **chiave** di R_2 :

$$0 \leq |R_1 \bowtie R_2| \leq |R_1|$$

perché i valori della chiave sono unici: ogni tupla di R_1 si combina con al più una tupla di R_2 (mentre una tupla di R_2 può combinarsi con più tuple di R_1);

- se il join coinvolge una chiave di R_2 e c'è un **vincolo di integrità referenziale** fra gli attributi di R_1 e la chiave di R_2 :

$$|R_1 \bowtie R_2| = |R_1|$$

perché ogni tupla di R_1 è associata a esattamente una tupla di R_2 .

Es.: $R_1(\text{Employee}, \text{Dept})$ e $R_2(\text{Room}, \text{Dept})$ con 3 tuple ciascuna, dove **Dept** non è chiave: il join può arrivare a $3 \times 3 = 9$ tuple (con due impiegati in B e due stanze in B se ne ottengono 4).

3.10 Join esterno

Nel join “normale” (*interno*) le tuple senza corrispondenza vengono scartate. Il **join esterno** (*outer join*) le conserva, completandole con valori **NULL**. Ne esistono tre varianti:

Sinistro ($\bowtie$)

mantiene tutte le tuple del *primo* operando, anche se non hanno corrispondenza nel secondo; gli attributi mancanti valgono **NULL**.

Destro ($\bowtie$)

mantiene tutte le tuple del *secondo* operando: $A \bowtie B$ equivale a $B \bowtie A$.

Completo ($\bowtie$)

mantiene tutte le tuple di entrambi: è l'unione del sinistro e del destro.

Employee		Department	
Employee	Dept	Dept	Chief
Rossi	A	B	Mori
Neri	B	C	Bruni
Bianchi	B		

Employee $\bowtie$ Department		
Employee	Dept	Chief
Neri	B	Mori
Bianchi	B	Mori
Rossi	A	NULL
NULL	C	Bruni

Il join sinistro contiene le prime tre righe, il destro le prime due e l'ultima.

3.11 Prodotto cartesiano e theta-join

Prodotto cartesiano è il join naturale fra due relazioni *senza attributi in comune*: ogni tupla del primo operando si combina con ogni tupla del secondo, quindi la cardinalità del risultato è il prodotto delle cardinalità. In pratica è utile solo se seguito da una selezione:

$$\sigma_{\text{condition}}(R_1 \bowtie R_2)$$

Theta-join questa combinazione ha un nome e una notazione propri:

$$R_1 \bowtie_{\text{condition}} R_2 = \sigma_{\text{condition}}(R_1 \bowtie R_2)$$

La condizione è di solito una congiunzione (**AND**) di atomi di confronto $A_1 \mathcal{R} A_2$, con $\mathcal{R}$ operatore di confronto ($=, >, <, \dots$).

Equi-join un theta-join in cui tutti gli atomi sono uguaglianze. È molto più usato del theta-join generale: permette di indicare su quali attributi fare il join senza ricorrere a ridenominazioni.

Es. con **Employee**(Employee, Dept) e **Department**(Code, Chief):

$$\text{Employee} \bowtie_{\text{Dept}=\text{Code}} \text{Department}$$

ha schema (Employee, Dept, Code, Chief). Il risultato è analogo a quello del join naturale con ridenominazione, che però ha un attributo in meno:

$$\text{Employee} \bowtie \rho_{\text{Dept} \leftarrow \text{Code}}(\text{Department}) \quad \text{schema}(\text{Employee}, \text{Dept}, \text{Chief})$$

3.12 Esempi di interrogazioni

Employee				Supervisor	
Number	Name	Age	Wage	Employee	Chief
7309	Rossi	34	45	7309	5698
5998	Bianchi	37	38	5998	5698
9553	Neri	42	35	9553	4076
5698	Bruni	43	42	5698	4076
4076	Mori	45	50	4076	8123
8123	Lupi	46	60		

1. Numero, nome, età e stipendio degli impiegati che guadagnano più di 40:

$$\sigma_{\text{Wage} > 40}(\text{Employee})$$

2. Numero, nome ed età degli impiegati che guadagnano più di 40:

$$\pi_{\text{Number, Name, Age}}(\sigma_{\text{Wage} > 40}(\text{Employee}))$$

3. Capi degli impiegati che guadagnano più di 40:

$$\pi_{\text{Chief}}(\text{Supervisor} \bowtie_{\text{Employee=Number}} \sigma_{\text{Wage} > 40}(\text{Employee}))$$

(risultato: 5698, 4076, 8123).

4. Nome e stipendio dei capi degli impiegati che guadagnano più di 40:

$$\pi_{\text{Name, Wage}}(\text{Employee} \bowtie_{\text{Number=Chief}} \pi_{\text{Chief}}(\text{Supervisor} \bowtie_{\text{Employee=Number}} \sigma_{\text{Wage} > 40}(\text{Employee})))$$

5. Impiegati che guadagnano più del proprio capo, con numero, nome e stipendio di entrambi. **Employee** serve due volte (come impiegato e come capo), quindi una copia va ridenominata:

$$\pi_{\text{Number, Name, Wage, NoC, NameC, WageC}}(\sigma_{\text{Wage} > \text{WageC}}(\pi_{\text{NoC, NameC, WageC, AgeC} \leftarrow \text{Number, Name, Wage, Age}}(\text{Employee}) \bowtie_{\text{NoC=Chief}} (\text{Supervisor} \bowtie_{\text{Employee=Number}} \text{Employee})))$$

(risultato: Rossi, 45, con capo Bruni, 42).

6. Capi i cui impiegati guadagnano **tutti** più di 40. Il “per ogni” si esprime con la *differenza*: tutti i capi meno quelli che hanno almeno un impiegato che guadagna ≤ 40 :

$$\pi_{\text{Chief}}(\text{Supervisor}) - \pi_{\text{Chief}}(\text{Supervisor} \bowtie_{\text{Employee=Number}} \sigma_{\text{Wage} \leq 40}(\text{Employee}))$$

(risultato: 8123).

3.13 Equivalenza di espressioni

Due espressioni E_1 ed E_2 sono **equivalenti** se producono lo stesso risultato *qualunque sia* l'istanza della base di dati. Le regole di equivalenza sono importanti perché i DBMS riscrivono le interrogazioni in espressioni equivalenti ma più efficienti.

1. **Atomizzazione delle selezioni** (cascata di selezioni):

$$\sigma_{A>20}(\sigma_{B='Jones'}(R)) = \sigma_{A>20 \wedge B='Jones'}(R)$$

2. **Idempotenza delle proiezioni** (X, Y attributi di R):

$$\pi_X(R) = \pi_X(\pi_{XY}(R))$$

3. **Anticipazione della selezione** rispetto al join (*push selections*), con A attributo di R_2 :

$$\sigma_{A=10}(R_1 \bowtie R_2) = R_1 \bowtie \sigma_{A=10}(R_2)$$

Riduce drasticamente la dimensione del risultato intermedio (e quindi il costo dell'operazione).

4. **Anticipazione della proiezione** rispetto al join (*push projections*), con $R_1(X_1)$, $R_2(X_2)$ e $Y_2 \subseteq X_2$:

$$\pi_{X_1 Y_2}(R_1 \bowtie R_2) = R_1 \bowtie \pi_{Y_2}(R_2)$$

Vale se $Y_2 \supseteq X_1 \cap X_2$, cioè se gli attributi eliminati ($X_2 - Y_2$) non sono coinvolti nel join.

5. **Definizione di theta-join**:

$$\sigma_{A=10}(R_1 \bowtie R_2) = R_1 \bowtie_{A=10} R_2$$

$$\sigma_{A=10 \wedge B='Jones'}(R_1 \bowtie R_2) = \sigma_{A=10}(R_1 \bowtie_{B='Jones'} R_2) = \sigma_{B='Jones'}(R_1 \bowtie_{A=10} R_2)$$

6–8. **Distributività**:

$$\sigma_{A=10}(R_1 \cup R_2) = \sigma_{A=10}(R_1) \cup \sigma_{A=10}(R_2)$$

$$\sigma_{A=10}(R_1 - R_2) = \sigma_{A=10}(R_1) - \sigma_{A=10}(R_2)$$

$$\pi_X(R_1 \cup R_2) = \pi_X(R_1) \cup \pi_X(R_2)$$

Attenzione: la proiezione *non* è distributiva rispetto alla differenza.

9–11. **Selezione e operatori insiemistici**:

$$\sigma_{A=10 \vee B='Jones'}(R) = \sigma_{A=10}(R) \cup \sigma_{B='Jones'}(R)$$

$$\sigma_{A=10 \wedge B='Jones'}(R) = \sigma_{A=10}(R) \cap \sigma_{B='Jones'}(R)$$

$$\sigma_{A=10 \wedge \neg(B='Jones')}(R) = \sigma_{A=10}(R) - \sigma_{B='Jones'}(R)$$

12. **Distributività del join rispetto all'unione**:

$$R \bowtie (R_1 \cup R_2) = (R \bowtie R_1) \cup (R \bowtie R_2)$$

Esempio di ottimizzazione numeri dei capi degli impiegati con meno di 30 anni (No = Number, Emp = Employee):

$$\pi_{\text{Chief}}(\sigma_{\text{No}=\text{Emp} \wedge \text{Age}<30}(\text{Employee} \bowtie \text{Supervisor}))$$

- regola 1, si spezza la selezione:

$$\pi_{\text{Chief}}(\sigma_{\text{No}=\text{Emp}}(\sigma_{\text{Age}<30}(\text{Employee} \bowtie \text{Supervisor})))$$

- regola 5 per No = Emp e regola 3 per Age < 30:

$$\pi_{\text{Chief}}(\sigma_{\text{Age}<30}(\text{Employee}) \bowtie_{\text{No}=\text{Emp}} \text{Supervisor})$$

- regola 4, si eliminano gli attributi inutili:

$$\pi_{\text{Chief}}(\pi_{\text{No}}(\sigma_{\text{Age}<30}(\text{Employee})) \bowtie_{\text{No}=\text{Emp}} \text{Supervisor})$$

Nota nel corso non è richiesto scrivere interrogazioni efficienti: conta che siano *corrette e chiare*, perché all'ottimizzazione pensa il DBMS.

3.14 Selezione con valori nulli

Una condizione atomica come $\text{Age} > 40$ è soddisfatta solo da valori *non nulli*: una tupla con $\text{Age} = \text{NULL}$ non compare nel risultato di $\sigma_{\text{Age} > 40}(\text{Employee})$.

Conseguenze indesiderate selezioni e condizioni atomiche sono valutate separatamente, quindi in presenza di NULL

$$\sigma_{\text{Age} > 30}(\text{People}) \cup \sigma_{\text{Age} \leq 30}(\text{People}) \neq \text{People} \quad \sigma_{\text{Age} > 30 \vee \text{Age} \leq 30}(\text{People}) \neq \text{People}$$

IS NULL / IS NOT NULL per riferirsi ai valori nulli si usano condizioni atomiche apposite (si potrebbe definire una logica a tre valori *true*, *false*, *unknown*, ma non è necessario):

$$\begin{aligned} & \sigma_{\text{Age} > 30}(\text{People}) \cup \sigma_{\text{Age} \leq 30}(\text{People}) \cup \sigma_{\text{Age IS NULL}}(\text{People}) \\ &= \sigma_{\text{Age} > 30 \vee \text{Age} \leq 30 \vee \text{Age IS NULL}}(\text{People}) = \text{People} \end{aligned}$$

Es.: la selezione

$$\sigma_{(\text{Age} > 40) \vee (\text{Age IS NULL})}(\text{Employee})$$

restituisce anche l'impiegato di cui non si conosce l'età.

3.15 Viste

Le viste sono rappresentazioni diverse degli stessi dati: corrispondono allo *schema esterno* dell'architettura ANSI/SPARC.

Relazioni di base

hanno contenuto autonomo (originale).

Relazioni derivate

il loro contenuto è definito tramite un'interrogazione; possono essere definite anche a partire da altre relazioni derivate (ma senza ricorsione).

Esempio con $\text{Concern}(\text{Employee}, \text{Dept})$ e $\text{Manage}(\text{Dept}, \text{Chief})$:

$$\text{Supervisor} = \pi_{\text{Employee}, \text{Chief}}(\text{Concern} \bowtie \text{Manage})$$

Due tipi di relazioni derivate

Viste materializzate

la relazione derivata è memorizzata fisicamente (in modo temporaneo o permanente).

- pro: i dati sono immediatamente disponibili per le interrogazioni;
- contro: ridondanza, aggiornamenti rallentati, raramente supportate dai DBMS.

Relazioni virtuali (o viste)

supportate da tutti i DBMS. Un'interrogazione su una vista viene trasformata in un'interrogazione sulla base di dati sottostante, sostituendo al nome della vista la sua definizione:

$$\sigma_{\text{Chief} = \text{'Leoni'}}(\text{Supervisor}) \rightsquigarrow \sigma_{\text{Chief} = \text{'Leoni'}}(\pi_{\text{Employee}, \text{Chief}}(\text{Concern} \bowtie \text{Manage}))$$

Perché le viste

- *schema esterno*: ogni utente vede ciò che gli interessa, nel modo in cui gli interessa, senza distrazioni, e solo ciò che è autorizzato a vedere;
- *strumento di programmazione*: semplificano la scrittura di interrogazioni complesse, soprattutto se ci sono sottoespressioni ripetute;
- permettono di usare programmi esistenti su schemi ristrutturati.

Le viste però *non* influiscono sull'efficienza delle interrogazioni.

Viste come strumento di programmazione impiegati che hanno lo stesso capo di Jones.

- senza vista:

$$\pi_{\text{Employee}}((\text{Concern} \bowtie \text{Manage}) \bowtie \rho_{\text{EmpR}, \text{DeptR} \leftarrow \text{Employee}, \text{Dept}}(\sigma_{\text{Employee}='Jones'}(\text{Concern} \bowtie \text{Manage})))$$

- con la vista **Supervisor**:

$$\pi_{\text{Employee}}(\text{Supervisor} \bowtie \rho_{\text{EmpR} \leftarrow \text{Employee}}(\sigma_{\text{Employee}='Jones'}(\text{Supervisor})))$$

Aggiornamento delle viste “aggiornare una vista” significa modificare le relazioni di base in modo che la vista, ricalcolata, rifletta l'aggiornamento. A ogni aggiornamento sulla vista deve corrispondere un aggiornamento sulle relazioni di base, ma questo può *non essere univoco*. Es.: inserire in **Supervisor** il fatto che Bruni è capo di Lupi è ambiguo se Bruni dirige due dipartimenti (in quale va Lupi?); inserire che Falchi è capo di Belli è impossibile senza inventare un dipartimento diretto da Falchi. Per questo sulle viste sono ammessi solo pochi tipi di aggiornamento.

3.16 Notazione alternativa per i join

È l'approccio adottato di solito in SQL:

- si ignora il join naturale (non si assumono implicitamente condizioni sugli attributi con lo stesso nome);
- con la sintassi **Relazione.Attributo** si distinguono gli attributi omonimi di relazioni diverse;
- le relazioni si “rinominano” definendo viste (assegnazione :=), e gli attributi si ridenominano solo quando serve per l'unione.

Es. (impiegati che guadagnano più del proprio capo), con $\text{Chief} := \text{Emp}$:

$$\pi_{\text{Emp.No}, \text{Emp.Name}, \text{Emp.Wage}, \text{Chief.No}, \text{Chief.Name}, \text{Chief.Wage}}(\sigma_{\text{Emp.Wage} > \text{Chief.Wage}}(\text{Chief} \bowtie_{\text{Chief.No} = \text{Sup.Chief}} (\text{Sup} \bowtie_{\text{Sup.Emp} = \text{Emp.No}} \text{Emp})))$$

3.17 Calcolo relazionale

Famiglia di linguaggi **dichiarativi** basati sulla logica del primo ordine. Due versioni:

- calcolo relazionale **su domini**;
- calcolo **su tuple con dichiarazioni di range**.

Negli esempi si usano le relazioni

Employee(Number, Name, Age, Salary) **Supervisor**(Chief, Employee)

3.18 Calcolo su domini

Un'espressione ha la forma

$$\{ A_1 : x_1, \dots, A_k : x_k \mid f \}$$

- f è una **formula** (con operatori booleani e quantificatori);
- $A_1 : x_1, \dots, A_k : x_k$ è la **target list**: $A_1, \dots, A_k$ sono attributi distinti (anche non presenti nella base di dati), $x_1, \dots, x_k$ variabili distinte.

Semantica: il risultato è una relazione su $A_1, \dots, A_k$ che contiene le tuple di valori per $x_1, \dots, x_k$ che rendono vera f .

Differenze rispetto al calcolo dei predicati

- i simboli di predicato corrispondono alle relazioni della base di dati, più i predicati “standard” predefiniti ($=, >, \dots$);
- non ci sono simboli di funzione;
- interessano soprattutto le formule *aperte* (con variabili libere);
- la notazione è *non posizionale*.

Esempi

0a. Numero, nome, età e stipendio di chi guadagna più di 40 ($\sigma_{\text{Salary} > 40}(\text{Employee})$):

$$\{ \text{Number} : m, \text{Name} : n, \text{Age} : a, \text{Salary} : s \mid \\ \text{Employee}(\text{Number} : m, \text{Name} : n, \text{Age} : a, \text{Salary} : s) \wedge s > 40 \}$$

0b. Numero, nome ed età di tutti gli impiegati ($\pi_{\text{Number}, \text{Name}, \text{Age}}(\text{Employee})$):

$$\{ \text{Number} : m, \text{Name} : n, \text{Age} : a \mid \\ \exists s (\text{Employee}(\text{Number} : m, \text{Name} : n, \text{Age} : a, \text{Salary} : s)) \}$$

Il quantificatore esistenziale sulle variabili che non compaiono nella target list si può sottintendere.

1. Numero, nome ed età di chi guadagna più di 40:

$$\{ \text{Number} : m, \text{Name} : n, \text{Age} : a \mid \\ \text{Employee}(\text{Number} : m, \text{Name} : n, \text{Age} : a, \text{Salary} : s) \wedge s > 40 \}$$

2. Capi degli impiegati che guadagnano più di 40 (il join si esprime usando la *stessa variabile* e nei due predicati):

$$\{ \text{Chief} : c \mid \text{Supervisor}(\text{Chief} : c, \text{Employee} : e) \\ \wedge \text{Employee}(\text{Number} : e, \text{Name} : n, \text{Age} : a, \text{Salary} : s) \wedge s > 40 \}$$

3. Nome e stipendio dei capi degli impiegati che guadagnano più di 40:

$$\{ \text{NameC} : nc, \text{WageC} : sc \mid \text{Employee}(\text{Number} : m, \text{Name} : n, \text{Age} : a, \text{Salary} : s) \wedge s > 40 \\ \wedge \text{Supervisor}(\text{Chief} : c, \text{Employee} : m) \\ \wedge \text{Employee}(\text{Number} : c, \text{Name} : nc, \text{Age} : ac, \text{Salary} : sc) \}$$

4. Impiegati che guadagnano più del proprio capo:

$$\begin{aligned} & \{ \text{No} : m, \text{Name} : n, \text{Wage} : s, \text{NoC} : c, \text{NameC} : nc, \text{WageC} : sc \mid \\ & \quad \text{Employee}(\text{Number} : m, \text{Name} : n, \text{Age} : a, \text{Salary} : s) \\ & \quad \wedge \text{Supervisor}(\text{Chief} : c, \text{Employee} : m) \\ & \quad \wedge \text{Employee}(\text{Number} : c, \text{Name} : nc, \text{Age} : ac, \text{Salary} : sc) \\ & \quad \wedge s > sc \} \end{aligned}$$

5. Numero e nome dei capi i cui impiegati guadagnano *tutti* più di 40:

$$\begin{aligned} & \{ \text{Number} : c, \text{Name} : n \mid \text{Employee}(\text{Number} : c, \text{Name} : n, \text{Age} : e, \text{Salary} : s) \\ & \quad \wedge \text{Supervisor}(\text{Chief} : c, \text{Employee} : m) \\ & \quad \wedge \neg \exists m' (\exists n' (\exists e' (\exists s' (\\ & \quad \quad \text{Employee}(\text{Number} : m', \text{Name} : n', \text{Age} : e', \text{Salary} : s') \\ & \quad \quad \wedge \text{Supervisor}(\text{Chief} : c, \text{Employee} : m') \wedge s' \leq 40)))) \} \end{aligned}$$

Quantificatori e De Morgan esistenziale e universale sono intercambiabili:

$$\begin{aligned} \neg(A \wedge B) &= \neg A \vee \neg B & \neg(A \vee B) &= \neg A \wedge \neg B \\ \neg \exists x A &= \forall x \neg A & \neg \forall x A &= \exists x \neg A & \exists x A &= \neg \forall x \neg A \end{aligned}$$

Nell'esempio 5 la negazione dell'esistenziale si può quindi riscrivere come

$$\begin{aligned} & \forall m' (\forall n' (\forall e' (\forall s' (\\ & \quad \neg (\text{Employee}(\text{Number} : m', \text{Name} : n', \text{Age} : e', \text{Salary} : s') \\ & \quad \quad \wedge \text{Supervisor}(\text{Chief} : c, \text{Employee} : m')) \\ & \quad \vee s' > 40)))) \end{aligned}$$

Pregi e difetti

- pregio: è dichiarativo;
- difetto: è “verboso” (moltissime variabili);
- difetto: ammette espressioni senza senso, come

$$\{A : x \mid \neg R(A : x)\} \quad \{A : x, B : y \mid R(A : x)\} \quad \{A : x, B : y \mid R(A : x) \wedge y = y\}$$

il cui risultato dipende dal dominio delle variabili e non solo dall'istanza: sono espressioni **dipendenti dal dominio**, da evitare. In algebra relazionale non si possono scrivere, perché l'algebra è *indipendente dal dominio*.

Calcolo su domini e algebra sono “equivalenti”:

- per ogni espressione del calcolo indipendente dal dominio esiste un'espressione dell'algebra equivalente;
- per ogni espressione dell'algebra esiste un'espressione del calcolo equivalente e indipendente dal dominio.

3.19 Calcolo su tuple con dichiarazioni di range

Nasce per superare i limiti del calcolo su domini:

- ridurre il numero di variabili: le variabili denotano *tuple* (una variabile per ciascuna tupla coinvolta) anziché singoli valori;
- garantire che tutti i valori provengano dalla base di dati.

Sintassi

$$\{ TargetList \mid RangeList \mid Formula \}$$

- *RangeList* elenca le variabili libere della formula, indicando per ciascuna la relazione da cui proviene: $x(R)$;
- *TargetList* ha elementi della forma $Y : x.Z$ (oppure $x.Z$, o $x.*$ per tutti gli attributi);
- *Formula* è costruita con:
 - atomi di confronto $x.A \mathcal{R} c$ e $x.A \mathcal{R} y.B$;
 - connettivi logici;
 - quantificatori che associano un range alle variabili: $\exists x(R)(\dots)$, $\forall x(R)(\dots)$.

Esempi (le stesse interrogazioni del calcolo su domini)

0a. $\{ i.* \mid i(\text{Employee}) \mid i.\text{Salary} > 40 \}$

0b. $\{ i.(\text{Number}, \text{Name}, \text{Age}) \mid i(\text{Employee}) \mid \}$

1. $\{ i.(\text{Number}, \text{Name}, \text{Age}) \mid i(\text{Employee}) \mid i.\text{Salary} > 40 \}$
2. $\{ s.\text{Chief} \mid i(\text{Employee}), s(\text{Supervisor}) \mid i.\text{Number} = s.\text{Employee} \wedge i.\text{Salary} > 40 \}$
3. Nome e stipendio dei capi degli impiegati che guadagnano più di 40:

$$\{ \text{NameC}, \text{WageC} : i'.(\text{Name}, \text{Salary}) \mid \\ i'(\text{Employee}), s(\text{Supervisor}), i(\text{Employee}) \mid \\ i'.\text{Number} = s.\text{Chief} \wedge i.\text{Number} = s.\text{Employee} \wedge i.\text{Salary} > 40 \}$$

4. Impiegati che guadagnano più del proprio capo:

$$\{ i.(\text{Name}, \text{Number}, \text{Salary}), \text{NameC}, \text{NoC}, \text{WageC} : i'.(\text{Name}, \text{Number}, \text{Salary}) \mid \\ i'(\text{Employee}), s(\text{Supervisor}), i(\text{Employee}) \mid \\ i'.\text{Number} = s.\text{Chief} \wedge i.\text{Number} = s.\text{Employee} \wedge i.\text{Salary} > i'.\text{Salary} \}$$

5. Capi i cui impiegati guadagnano tutti più di 40:

$$\{ i.(\text{Number}, \text{Name}) \mid s(\text{Supervisor}), i(\text{Employee}) \mid i.\text{Number} = s.\text{Chief} \\ \wedge \neg(\exists i'(\text{Employee})(\exists s'(\text{Supervisor})(s.\text{Chief} = s'.\text{Chief} \\ \wedge s'.\text{Employee} = i'.\text{Number} \wedge i'.\text{Salary} \leq 40))) \}$$

Limite: l'unione questo calcolo non permette di esprimere alcune interrogazioni importanti, in particolare le **unioni** $R_1(AB) \cup R_2(AB)$: ogni variabile ha un solo range, mentre il risultato dovrebbe provenire da due relazioni diverse (né una né due variabili bastano). Intersezione e differenza invece si possono esprimere. Per questo SQL, che si basa su questo calcolo, ha un operatore esplicito di unione, mentre intersezione e differenza non sono presenti in tutte le implementazioni.

3.20 Limiti di algebra e calcolo

Algebra e calcolo sono sostanzialmente equivalenti ed esprimono un insieme significativo di operazioni, ma alcune interrogazioni utili restano fuori:

- si possono solo *estrarre* valori, non *calcolarne* di nuovi:
 - a livello di tupla (conversioni, somme, differenze, ...);
 - su insiemi di tuple (somme, medie, ...);

queste estensioni sono presenti in SQL;

- interrogazioni **ricorsive**, come la **chiusura transitiva**.

Chiusura transitiva data `Supervisor(Employee, Chief)`, trovare per ogni impiegato *tutti* i suoi superiori (il capo, il capo del capo, e così via).

		Employee Superior	
Employee	Chief		
Rossi	Lupi	Rossi	Lupi
Neri	Bruni	Neri	Bruni
Lupi	Falchi	Lupi	Falchi
Falchi	Leoni	Falchi	Leoni
		Rossi	Falchi (1 join)
		Lupi	Leoni (1 join)
		Rossi	Leoni (2 join)

Con un join della relazione con se stessa (più ridenominazione) si ottengono i superiori di secondo livello, con due join quelli di terzo, e così via: il numero di join necessari dipende dall'*istanza* e non è limitato a priori. In algebra relazionale quindi **non** è possibile scrivere un'unica espressione che calcoli la chiusura transitiva di una qualunque relazione binaria.

3.21 Datalog

Linguaggio di programmazione logica per basi di dati, derivato dal Prolog. Usa due tipi di predicati:

- **estensionali**: le relazioni della base di dati;
- **intensionali**: corrispondono alle viste, e sono definiti tramite **regole**.

Sintassi

$$head \leftarrow body$$

- *head* (testa) è un predicato atomico intensionale;
- *body* (corpo) è una lista (congiunzione) di predicati atomici;
- le interrogazioni sono predicati atomici preceduti da ?.

Esempi

- Impiegati di 30 anni (basta un predicato estensionale con una costante):

? Employee(Number : m , Name : n , Age : 30, Salary : s)

- Impiegati che guadagnano più di 40 (serve un predicato intensionale):

RichE(Number : m , Name : n , Age : e , Salary : s) $\leftarrow$
Employee(Number : m , Name : n , Age : e , Salary : s), $s > 40$
? RichE(Number : m , Name : n , Age : e , Salary : s)

- Numero, nome ed età di tutti gli impiegati (proiezione):

PublicInfos(Number : m , Name : n , Age : e) $\leftarrow$
Employee(Number : m , Name : n , Age : e , Salary : s)
? PublicInfos(Number : m , Name : n , Age : e)

- Capi degli impiegati che guadagnano più di 40 (join):

ChiefsOfRichE(Chief : c) $\leftarrow$
Employee(Number : m , Name : n , Age : e , Salary : s), $s > 40$,
Supervisor(Chief : c , Employee : m)
? ChiefsOfRichE(Chief : c)

- Capi i cui impiegati guadagnano *tutti* più di 40 (serve la **negazione**):

ChiefsOfNoRichE(Chief : c) $\leftarrow$
Supervisor(Chief : c , Employee : m),
Employee(Number : m , Name : n , Age : e , Salary : s), $s \leq 40$
ChiefsOfOnlyRichE(Number : c , Name : n) $\leftarrow$
Employee(Number : c , Name : n , Age : e , Salary : s),
Supervisor(Chief : c , Employee : m),
not ChiefsOfNoRichE(Chief : c)
? ChiefsOfOnlyRichE(Number : c , Name : n)

- Per ogni impiegato, tutti i suoi superiori (serve la **ricorsione**: è la chiusura transitiva):

HighGrade(Employee : i , SuperChief : c) $\leftarrow$ Supervisor(Employee : i , Chief : c)
HighGrade(Employee : i , SuperChief : c) $\leftarrow$
Supervisor(Employee : i , Chief : c'), HighGrade(Employee : c' , SuperChief : c)

Semantica e potere espressivo la definizione della semantica delle interrogazioni ricorsive è delicata, soprattutto in presenza di negazione.

- Datalog *non ricorsivo senza negazione* equivale al calcolo senza negazione e senza quantificatore universale;
- Datalog *non ricorsivo con negazione* equivale a calcolo e algebra;
- Datalog *ricorsivo senza negazione* e calcolo sono *incomparabili*;
- Datalog *ricorsivo con negazione* è più espressivo di calcolo e algebra.